Data Science II Homework 2

Yongyan Liu (yl6107)

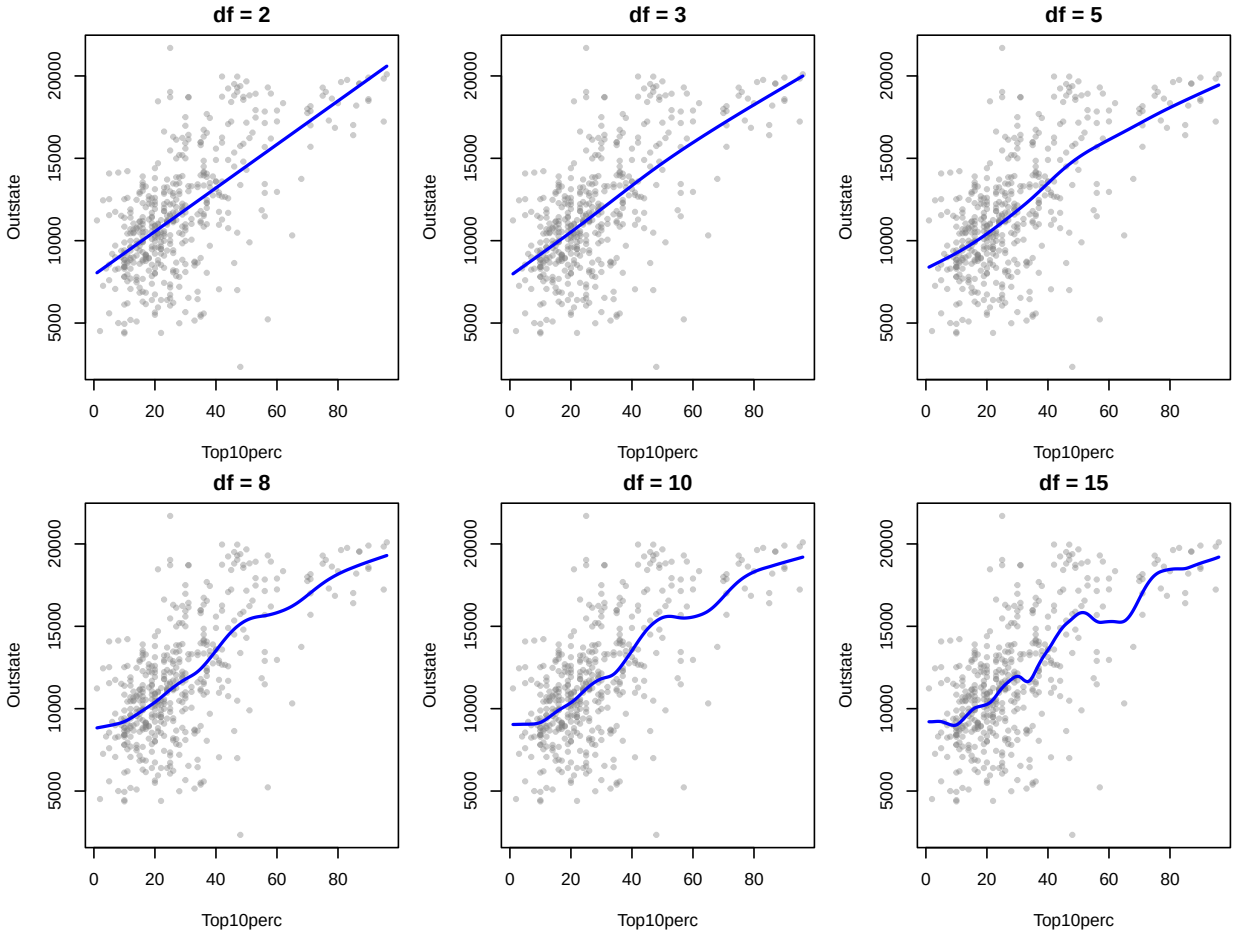
2026-03-06

Load the dataset and split the dataset for training and testing.

```
library(tidyverse)
library(rsample)
library(caret)
library(earth)
library(mgcv)
library(pdp)

# Load data (first column is college name, use as row names)
college <- read.csv("College.csv", row.names = 1)

# Train/test split (80/20)
set.seed(2026)
data_split <- initial_split(college, prop = 0.8)
train_data <- training(data_split)
test_data <- testing(data_split)
```

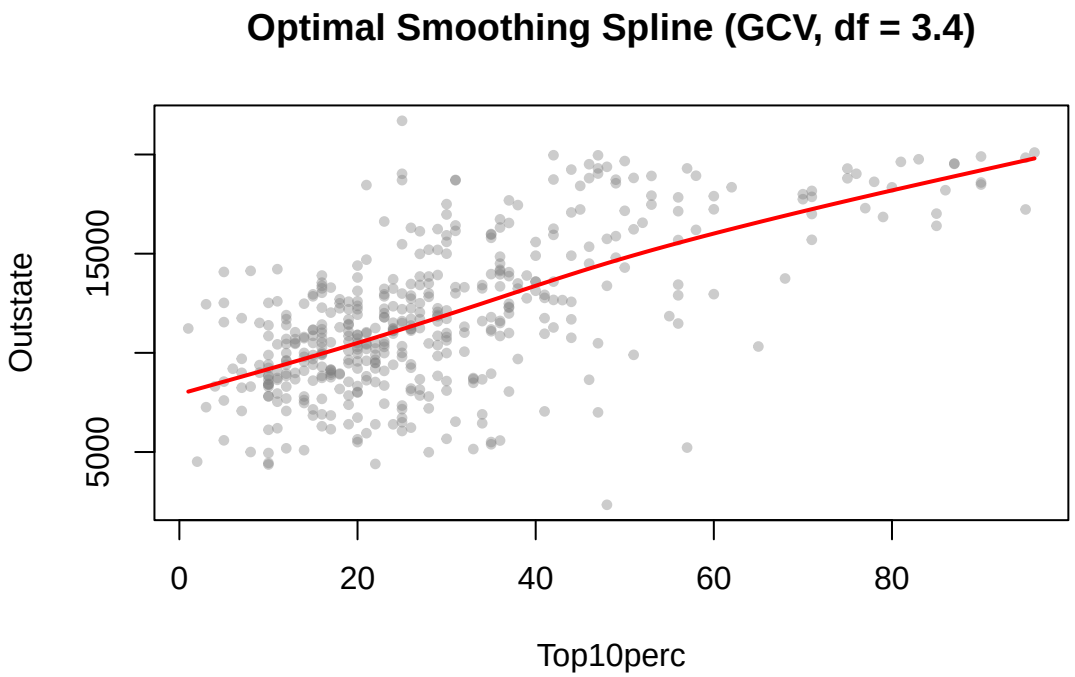
(a) Smoothing Splines

```
# Range of degrees of freedom to explore
df_values <- c(2, 3, 5, 8, 10, 15)

# Set up plot
x_grid <- seq(min(train_data$Top10perc), max(train_data$Top10perc), length = 200)

par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))
for (df_val in df_values) {
  fit_ss <- smooth.spline(train_data$Top10perc, train_data$Outstate, df = df_val)
  pred_ss <- predict(fit_ss, x = x_grid)

  plot(train_data$Top10perc, train_data$Outstate,
       col = rgb(0.5, 0.5, 0.5, 0.4), pch = 19, cex = 0.6,
       xlab = "Top10perc", ylab = "Outstate",
       main = paste("df =", df_val))
  lines(pred_ss, col = "blue", lwd = 2)
}
```



As df increases, the curve becomes more flexible: df = 2 is nearly linear, df = 3–5 captures the increasing trend with moderate curvature, and df = 8–15 overfits with unnecessary wiggles. We select the optimal df using GCV.

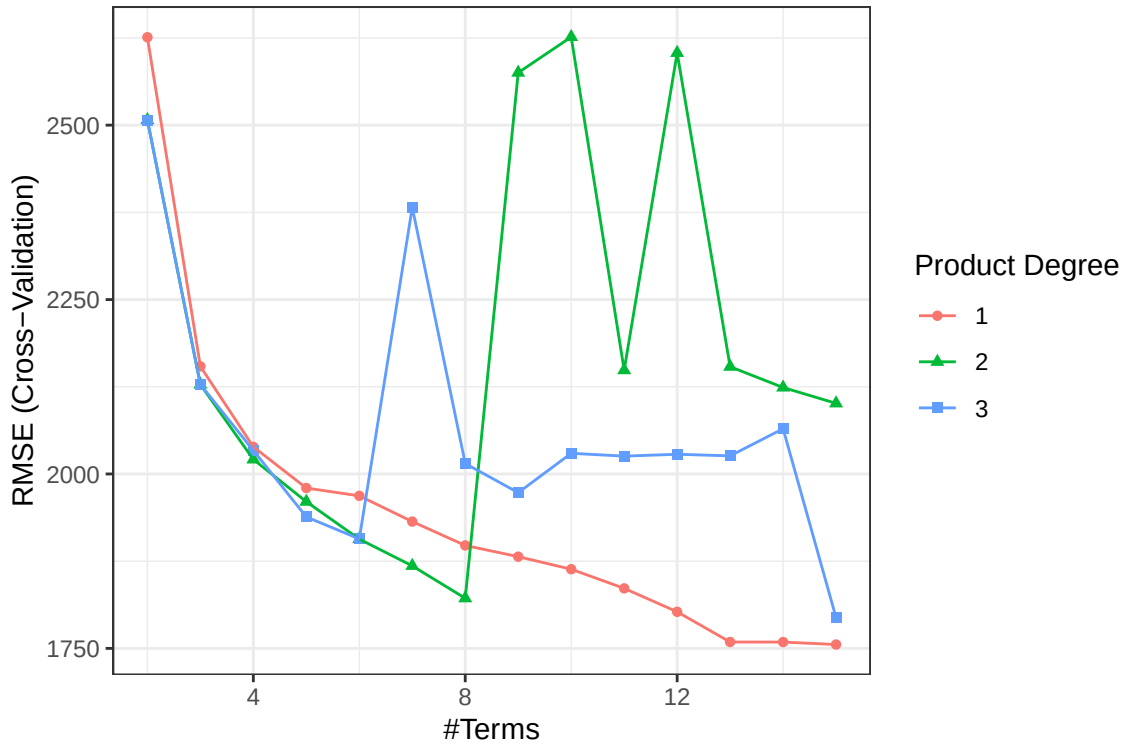
```r
# Fit with GCV (automatic df selection)
fit_opt <- smooth.spline(train_data$Top10perc, train_data$Outstate)
optimal_df <- fit_opt$df

# Plot the optimal fit
pred_opt <- predict(fit_opt, x = x_grid)

plot(train_data$Top10perc, train_data$Outstate,
     col = rgb(0.5, 0.5, 0.5, 0.4), pch = 19, cex = 0.6,
     xlab = "Top10perc", ylab = "Outstate",
     main = paste0("Optimal Smoothing Spline (GCV, df = ", round(optimal_df, 2), ")"))
lines(pred_opt, col = "red", lwd = 2)
```



The GCV-selected degrees of freedom is **3.4**, which balances bias and variance automatically.

(b) MARS Model

```
ctrl <- trainControl(method = "cv", number = 10, selectionFunction = "oneSE")

# Tune MARS: degree (1 = additive, 2 = two-way interactions) and nprune
mars_grid <- expand.grid(degree = 1:3,
                         nprune = 2:15)

set.seed(2026)
mars_fit <- train(Outstate ~ .,
                  data = train_data,
                  method = "earth",
                  tuneGrid = mars_grid,
                  trControl = ctrl)

ggplot(mars_fit) + theme_bw()
```




```
mars_fit$bestTune
```

```
##    nprune degree
## 11     12      1
```

```
# Final model coefficients (regression function)
coef(mars_fit$finalModel)
```

```
##          (Intercept)     h(Expend-15494)     h(83-Grad.Rate)  h(4130-Room.Board)
##        9498.1268053          -0.6717143         -26.4968529          -1.3464341
## h(F.Undergrad-1379) h(1379-F.Undergrad)    h(1350-Personal)         h(Apps-3847)
##          -0.3440150          -1.6702202           1.2707471           0.3926377
##   h(22-perc.alumni)      h(Expend-4933)      h(2314-Accept)       h(876-Enroll)
##         -61.2073333           0.7052391          -1.8556830           4.8211180
```

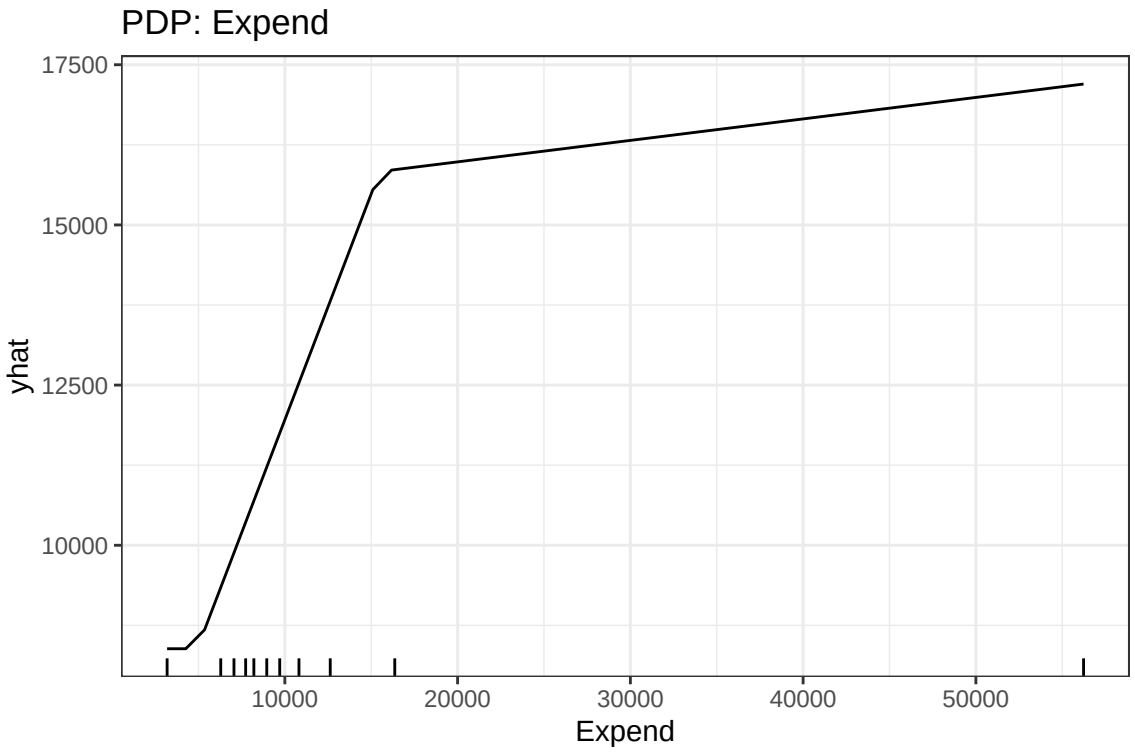
```
# Show the regression function
summary(mars_fit$finalModel)
```

```
## Call: earth(x=matrix[452,16], y=c(10200,7820,10...), keepxy=TRUE, degree=1,
##             nprune=12)
##
##                     coefficients
## (Intercept)            9498.1268
## h(Apps-3847)              0.3926
## h(2314-Accept)           -1.8557
## h(876-Enroll)             4.8211
## h(1379-F.Undergrad)      -1.6702
```

```
## h(F.Undergrad-1379)      -0.3440
## h(4130-Room.Board)       -1.3464
## h(1350-Personal)          1.2707
## h(22-perc.alumni)       -61.2073
## h(Expend-4933)            0.7052
## h(Expend-15494)          -0.6717
## h(83-Grad.Rate)         -26.4969
##
## Selected 12 of 29 terms, and 9 of 16 predictors (nprune=12)
## Termination condition: Reached nk 33
## Importance: Expend, Grad.Rate, Room.Board, Personal, Accept, F.Undergrad, ...
## Number of terms at each degree of interaction: 1 11 (additive model)
## GCV 2973671    RSS 1210790774    GRSq 0.7892719    RSq 0.8093293
```

Using the one-standard-error rule, the selected tuning parameters are degree = 1 and nprune = 12, which gives the simplest model whose CV error is within one SE of the minimum.

```
# Partial dependence plot for Expend
p1 <- pdp::partial(mars_fit, pred.var = "Expend", grid.resolution = 50) %>%
  autoplot(train = train_data, rug = TRUE) +
  ggtitle("PDP: Expend") +
  theme_bw()
p1
```



The PDP for `Expend` shows tuition increases sharply with expenditure up to a point, then levels off.

```
# Test error
mars_pred <- predict(mars_fit, newdata = test_data)
```

```r
mars_mse <- mean((mars_pred - test_data$Outstate)^2)
cat("MARS Test RMSE:", round(sqrt(mars_mse), 2), "\n")
```

```
## MARS Test RMSE: 1632.08
```

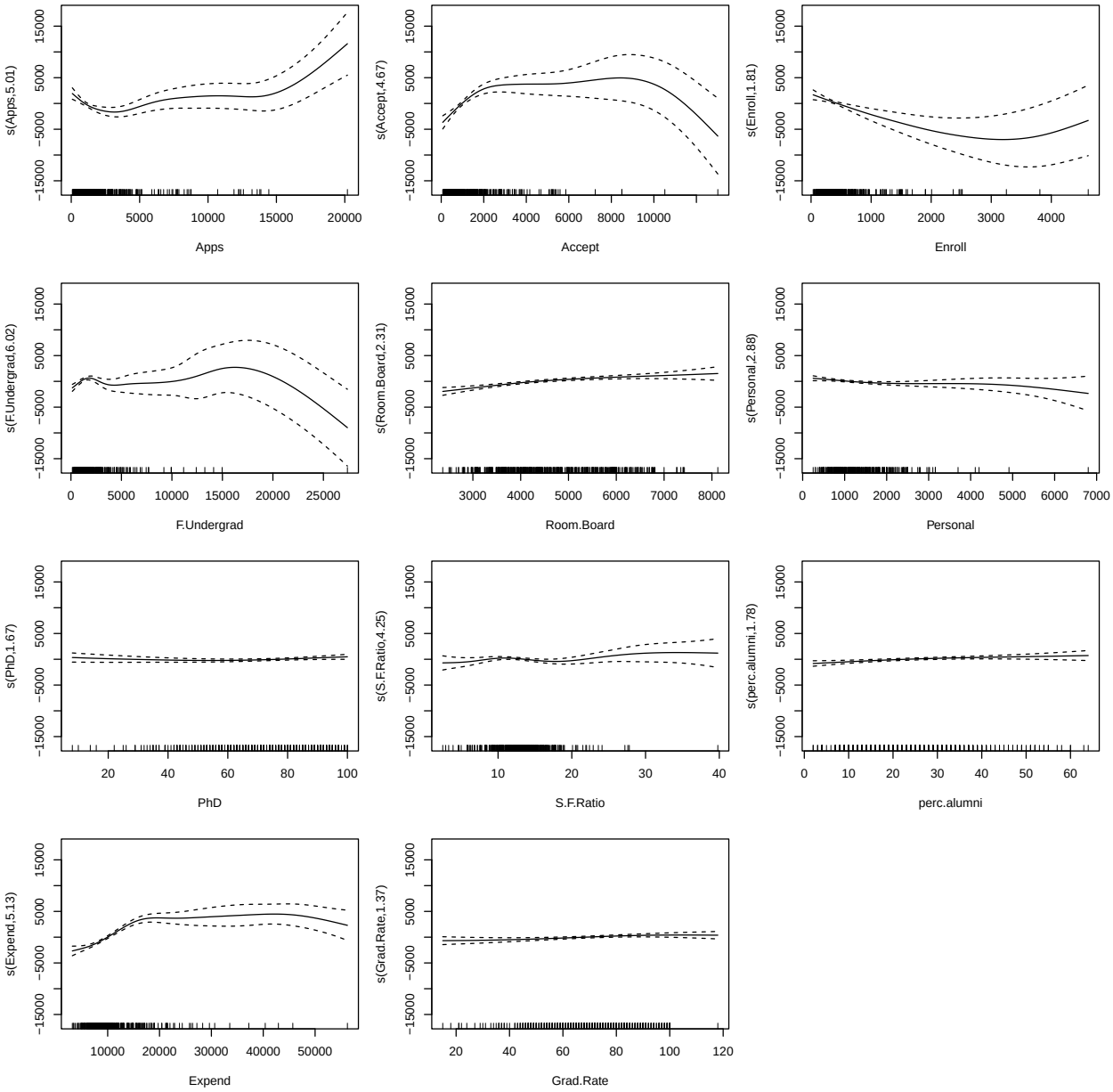
(c) Generalized Additive Model (GAM)

```r
# Fit GAM with smooth terms for all predictors
# select = TRUE adds extra shrinkage penalty to drop unimportant terms
gam_fit <- gam(Outstate ~ s(Apps) + s(Accept) + s(Enroll) + s(Top10perc) +
                 s(Top25perc) + s(F.Undergrad) + s(P.Undergrad) +
                 s(Room.Board) + s(Books) + s(Personal) + s(PhD) +
                 s(Terminal) + s(S.F.Ratio) + s(perc.alumni) +
                 s(Expend) + s(Grad.Rate),
               data = train_data, select = TRUE)

summary(gam_fit)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## Outstate ~ s(Apps) + s(Accept) + s(Enroll) + s(Top10perc) + s(Top25perc) +
##     s(F.Undergrad) + s(P.Undergrad) + s(Room.Board) + s(Books) +
##     s(Personal) + s(PhD) + s(Terminal) + s(S.F.Ratio) + s(perc.alumni) +
##     s(Expend) + s(Grad.Rate)
##
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 11779.54      75.68   155.6   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##                       edf Ref.df      F  p-value
## s(Apps)         5.010e+00      9  3.709 4.36e-06 ***
## s(Accept)       4.668e+00      9  4.859  < 2e-16 ***
## s(Enroll)       1.808e+00      9  2.752 2.13e-05 ***
## s(Top10perc)    7.497e-01      9  0.533 0.009687 **
## s(Top25perc)    8.441e-10      9  0.000 0.745297
## s(F.Undergrad)  6.024e+00      9  3.537 0.001053 **
## s(P.Undergrad)  3.142e-10      9  0.000 0.763084
## s(Room.Board)   2.306e+00      9  5.590  < 2e-16 ***
## s(Books)        6.282e-10      9  0.000 0.538428
## s(Personal)     2.878e+00      9  1.607 0.011475 *
## s(PhD)          1.669e+00      9  0.546 0.036365 *
## s(Terminal)     6.901e-10      9  0.000 0.320755
## s(S.F.Ratio)    4.254e+00      9  1.294 0.018602 *
## s(perc.alumni)  1.775e+00      9  1.433 0.000455 ***
```

```
## s(Expend)      5.132e+00      9 16.589  < 2e-16 ***
## s(Grad.Rate)   1.369e+00      9  1.040 0.000796 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Rank: 144/145
## R-sq.(adj) =  0.816   Deviance explained = 83.1%
## GCV = 2.8309e+06  Scale est. = 2.5889e+06  n = 452
```

```r
# Identify significant nonlinear terms (edf > 1 and p-value < 0.05)
gam_summary <- summary(gam_fit)
edf <- gam_summary$s.table[, "edf"]
pval <- gam_summary$s.table[, "p-value"]
nonlinear_idx <- which(edf > 1 & pval < 0.05)
nonlinear_names <- names(edf)[nonlinear_idx]

# Plot only the nonlinear smooth terms
par(mfrow = c(ceiling(length(nonlinear_idx) / 3), 3), mar = c(4, 4, 2, 1))
for (idx in nonlinear_idx) {
  plot(gam_fit, shade = TRUE, shade.col = "lightskyblue",
       seWithMean = TRUE, select = idx)
}
```



The plots above show the significant nonlinear smooth terms (edf > 1 and p-value < 0.05). Terms with edf close to 1 (essentially linear) or non-significant p-values are not shown.

```
# Test error
gam_pred <- predict(gam_fit, newdata = test_data)
gam_mse <- mean((gam_pred - test_data$Outstate)^2)
cat("GAM Test RMSE:", round(sqrt(gam_mse), 2), "\n")
```

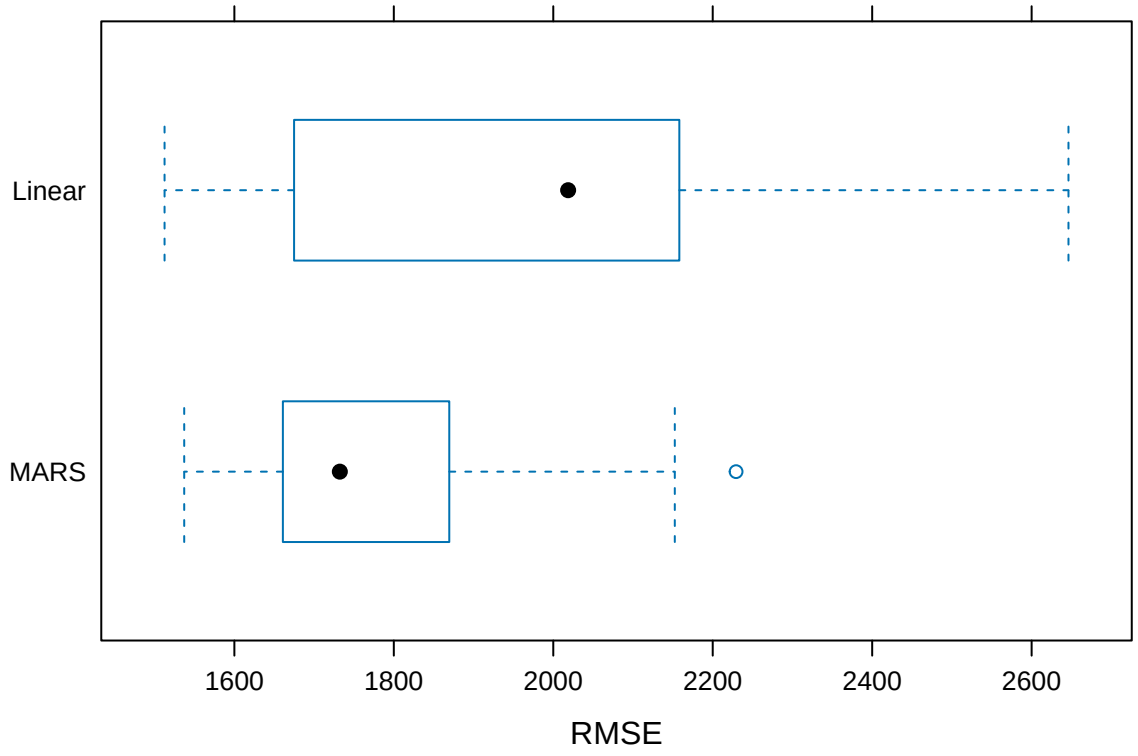
```
## GAM Test RMSE: 1635.91
```

(d) MARS vs. Linear Model for This Dataset

```
# Fit a linear model for comparison
set.seed(2026)
lm_fit <- train(Outstate ~ .,
                data = train_data,
                method = "lm",
                trControl = ctrl)

# Compare via resampling
resamp <- resamples(list(MARS = mars_fit, Linear = lm_fit))
summary(resamp)
```

```
##
## Call:
## summary.resamples(object = resamp)
##
## Models: MARS, Linear
## Number of resamples: 10
##
## MAE
##            Min.  1st Qu.   Median     Mean  3rd Qu.     Max. NA's
## MARS   1223.867 1277.660 1336.108 1359.107 1414.754 1592.573    0
## Linear 1284.488 1426.897 1603.148 1570.933 1673.210 1918.268    0
##
## RMSE
##            Min.  1st Qu.   Median    Mean  3rd Qu.     Max. NA's
## MARS   1537.112 1665.532 1732.295 1802.44 1857.866 2229.302    0
## Linear 1512.438 1740.137 2018.641 1998.37 2157.329 2646.231    0
##
## Rsquared
##            Min.   1st Qu.    Median      Mean   3rd Qu.      Max. NA's
## MARS   0.637717 0.7433477 0.7829231 0.7696856 0.8128305 0.8322908    0
## Linear 0.593595 0.6723527 0.7205281 0.7205204 0.7774713 0.8593534    0
```

```
bwplot(resamp, metric = "RMSE")
```




```
# Test errors
lm_pred <- predict(lm_fit, newdata = test_data)
lm_mse <- mean((lm_pred - test_data$Outstate)^2)

cat("Linear Model Test RMSE:", round(sqrt(lm_mse), 2), "\n")
```

```
## Linear Model Test RMSE: 1924.72
```

```
cat("MARS Test RMSE:", round(sqrt(mars_mse), 2), "\n")
```

```
## MARS Test RMSE: 1632.08
```

Yes, we favor MARS over the linear model. MARS achieves a lower test RMSE and captures the nonlinear relationships (e.g., `Expend`) that a linear model cannot. It also performs automatic variable selection while remaining interpretable.

(e) MARS vs. Linear Model in Practice

Neither is universally superior. MARS is preferred when nonlinearity is present and the goal is prediction, as it captures threshold effects and performs automatic variable selection. A linear model is preferred when the goal is inference, the sample size is small, or the true relationship is approximately linear. In practice, one should compare both via cross-validation and choose based on the prediction–interpretability trade-off.